DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

Academic Year 2026-2027
ODD SEMESTER

Skill-based Evaluation Report

(III Internal Assessment)

for

25AI2014
Agentic AI

in partial fulfillment for the award of the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE)

Submitted by
Demian Sharwin Xaxa (URK23CS7011)
Nithya Maria Mathew (URK23CS7004)

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved
NAAC A++ Accredited

October 2026

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved
NAAC A++ Accredited

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that the skill-based evaluation report for the course **25AI2014 Agentic AI** is a bonafide work done during the odd semester of the academic year 2026-2027 by

Demian Sharwin Xaxa (URK23CS7011)
Nithya Maria Mathew (URK23CS7004)

in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence) of Karunya Institute of Technology and Sciences.

Submitted for the assessment held on **29-10-2026**

Signature of the Class Teacher

Sophia Stephen
Assistant Professor
Division of Artificial Intelligence and Machine Learning

DECLARATION

We hereby declare that the skill-based evaluation report for the course **25AI2014 Agentic AI** submitted by us to Karunya Institute of Technology and Sciences is an authentic work carried out by us under the guidance of **Sophia Stephen** All the information presented in this report is original and has not been submitted for any other academic purpose.

We acknowledge that the sources of information used in this report have been appropriately cited and referenced. Any contributions from other individuals or organizations have been duly acknowledged.

We understand that any act of plagiarism or academic dishonesty is strictly prohibited and may result in disciplinary action as per the university's regulations.

We take full responsibility for the content and integrity of this report, and we are willing to provide further clarification or information if required.

Signature of student(s)

Demian Sharwin Xaxa

Nithya Maria Mathew

<

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

TYPE OF SKILLS AND THEIR REQUIREMENT

(The student team may select multiple skills and proceed accordingly with their work)

Select your choice	Type of Generic Skills	Requirement
✓	Programming Skills	Assessing proficiency in programming languages commonly used in engineering, such as Python, C, C++, Java, etc.,
	Laboratory Proficiency	Assessing the ability to conduct experiments accurately and efficiently in laboratory settings.
✓	Prototyping and Fabrication	Evaluating skills in building prototypes and fabricating engineering components using various tools and materials.
	Computer-Aided Design (CAD)	Measuring proficiency in using CAD software to design and model engineering components and systems.
	Software Kit/Tool Operation	Assessing competency in operating specialized engineering software effectively.
✓	Data Analysis	Measuring proficiency in collecting, analyzing, and interpreting data using statistical methods and software.
✓	Troubleshooting	Evaluating the ability to diagnose and solve technical issues and failures in engineering systems.
	Fieldwork and Industry visits	Assessing skills in conducting fieldwork, surveys, and site visits to gather data and assess real-world engineering challenges.
✓	Integration of Theory and Practice	Evaluating the ability to apply theoretical knowledge to practical engineering problems and projects effectively.
	Conference/Journal/Patent publication	Writing high-quality articles and submitting them for publication in Scopus-indexed conferences or journals involves careful preparation and rigorous review processes.

Signature of the Student(s)

Demian Sharwin Xaxa

Nithya Maria Mathew

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

EVALUATION REPORT

(The class teacher is responsible for assessing the student team during the designated evaluation period based on the following criteria.)

S.No	Evaluation Parameter	Max. Marks	Marks Secured
1	Agent Planning and Goal Formulation	4	
2	Agent Architecture and Integration	4	
3	Multi-Agent Orchestration	4	
4	Implementation and Functionality	4	
5	Testing and Performance Evaluation	4	
6	Innovation	4	
7	Presentation and Viva	4	
8	Report Preparation	4	

Signature of the Class Teacher

Sophia Stephen
Assistant Professor
Division of Artificial Intelligence and Machine Learning

TABLE OF CONTENTS

S.No	Content Title	Page No
1	**Introduction**	**9**
1.1	Background and motivation	9
1.2	Problem statement	9
1.3	Objectives	9
1.4	Scope and constraints	9
1.5	Skill sets demonstrated	10
1.6	Organisation of the report	10
2	**Planning and preparation**	**11**
2.1	Goal formulation	11
2.2	Requirements	11
2.3	Technology and tool selection	11
2.4	Dataset planning	12
2.5	Work plan and timeline	12
2.6	Design principles	13
2.7	Risk plan	13
3	**Key components of the identified skill sets**	**15**
3.1	System architecture	15
3.2	Agent roles and responsibilities	15
3.3	Multi-agent orchestration	16
3.4	Synthetic data generation and attack storyline	19
3.5	Event normalisation (Ingest agent)	20
3.6	Correlation agent and attack chain	20
3.7	Fact builder	21
3.8	Narrator agent	21
3.9	Verifier and independent judge	22
3.10	Report builder	23
3.11	Implementation summary	23
3.12	Mapping to the evaluation parameters	24
4	**Proofs of outcome and feedback**	**25**
4.1	Testing	25
4.2	Evaluation design	25
4.3	Results	26
4.4	Proof of outcome: the generated report	28
4.5	Troubleshooting and issues resolved	30
4.6	Feedback and actions taken	31
4.7	Limitations and threats to validity	32
4.8	Conclusion and future work	32
5	**Powerpoint Presentation**	**33**
5.1	Slide outline	33
5.2	Demonstration plan (seven minutes)	33
5.3	Anticipated questions	34
6	**Certification**	**35**
6.1	Supporting evidence	35
6.2	Certificates	35
6.3	Statement on data and tool use	35

LIST OF FIGURES

Figure	Title	Page No
1	End-to-end architecture of EvidenceWeave	15
2	Agent orchestration: controller, eight agents, and the shared artifact store	17
3	Sequence of one verification and repair cycle	18
4	Attack storyline of case02 as logged (UTC), with the decoy lane	19
5	Attack chain of case02: how findings link, and what stays out	21
6	The four verification layers and their failure codes	22
7	Attack steps and attacker entities present in the final narrative (mean of four cases)	26
8	Report header, summary tiles and timeline for case02	29
9	Executive summary and the brute-force stage card with facts, verified sentences and cite chips	30

LIST OF TABLES

Table	Title	Page No
1	Scope and constraints of the project	9
2	Skills addressed by the project	10
3	Goal decomposition and success criteria	11
4	Functional and non-functional requirements	11
5	Tools and technologies	11
6	Generated cases used in the project	12
7	Phase plan and outputs	13
8	Risks and planned mitigations	13
9	The eight agents	16
10	Orchestration design decisions	17
11	Attack stages in case02	19
12	Normalised event schema	20
13	Correlation rules and default thresholds	20
14	Verification layers	22
15	Command-line interface	23
16	Where each evaluation parameter is addressed	24
17	Passing tests after each phase	25
18	What the tests cover	25
19	Arms and metrics	25
20	Chain detection on the four evaluation cases	26
21	Cost of the approaches	27
22	Raw-mode versus fact-sheet narration (Phase 5b, development cases)	27
23	Held-out corruption test (pooled over two cases)	27
24	Audit labels versus the verifier decision	28
25	Defects seen in the first model output and the layer that handled them	30
26	Issues found during the project and how they were fixed	30
27	Feedback and resulting actions	31
28	Presentation outline	33
29	Timed demonstration script	33
30	Questions and short answers	34
31	Evidence available for verification	35
32	Certificate record	35

1. INTRODUCTION

1.1 Background and motivation

When a server is compromised, investigators receive evidence from several systems at once: Linux authentication logs, web-server access logs and Windows event logs. Each source has its own format, its own clock and thousands of routine lines that hide the few lines that matter. Reconstructing the attack means reading all of them together, linking events across hosts and writing down what happened in a way that another person can check.

Large language models can summarise logs fluently, but in forensic work fluency is not enough. A model may invent an address, change a time, or claim an action that no log line shows. A single unsupported sentence in an incident report can mislead an investigation. The question behind this project is therefore not "can an LLM describe an attack?" but "how can a small local model be allowed to describe an attack only when every statement can be traced to evidence?".

1.2 Problem statement

To design, build and evaluate **EvidenceWeave**, an autonomous log-correlation system that reads mixed log sources, reconstructs the multi-stage attack chain, and produces a plain-English incident narrative in which every sentence is either verified against the cited log lines or removed. The whole system must be free and local: it may use only open models served by Ollama on a laptop and no paid API.

1.3 Objectives

- Read three log formats (Linux auth.log, nginx access log, Windows event JSON Lines) and convert every line into one event with a traceable identifier such as `AUTH-L342`.
- Detect a six-stage attack chain with deterministic rules, deriving the attacker address, user and payload URL from the data instead of hard-coding them.
- Keep decoys and unrelated events out of the chain, and report missing evidence honestly instead of guessing.
- Use a local language model only to explain facts that the rules have already established.
- Verify every generated sentence with deterministic checks and an independent second model, repair what can be repaired and drop the rest.
- Deliver the result as one offline HTML report with clickable evidence.
- Measure the system against known ground truth, including a baseline, a held-out hallucination test and a manual-style audit.

1.4 Scope and constraints

Table 1: Scope and constraints of the project

Aspect	Decision
Log sources	Linux authentication log, nginx combined access log, Windows security events as JSON Lines.
Models	Local only through Ollama: qwen2.5:7b-instruct (narrator) and llama3.1:8b (judge). No paid API.
Data	Synthetic cases generated with a seed, each with a hidden answer key. No real personal or organisational data is used.
Language and platform	Python 3.12 on Windows, command-line tool plus a self-contained HTML report.
Out of scope	Real-time monitoring, memory or disk images, real-world log collections, automatic correction of clock skew.

1.5 Skill sets demonstrated

The skill-based evaluation allows several generic skills to be selected. The work in this report exercises the following ones; the later sections show where each appears.

Table 2: Skills addressed by the project

Skill	How it is demonstrated
Programming Skills	A modular Python package (generator, parsers, correlation, narrator, verifier, report), a command-line interface and 212 automated tests.
Data Analysis	Parsing and normalising 3,907 events in the hard case, rule-based correlation, precision and recall against ground truth, Wilson confidence intervals.
Software Kit/Tool Operation	Ollama with two local models, git and GitHub, pytest, a deterministic HTML report generator.
Troubleshooting	Diagnosis of 47 failing sentences, clock-skew handling, a hung model request, and nine other issues recorded in Section 4.
Integration of Theory and Practice	Digital-forensics principles (traceable evidence, reproducibility, no overstated conclusions) applied through agent orchestration and verification.

1.6 Organisation of the report

Section 2 describes the planning and preparation. Section 3 describes the key components, including the architecture and the agent orchestration. Section 4 gives the proofs of outcome, the evaluation results and the feedback that shaped the design. Section 5 outlines the presentation and demonstration. Section 6 lists the supporting evidence and certification items.

2. PLANNING AND PREPARATION

2.1 Goal formulation

The top-level goal of the system is: **given a folder of raw logs, produce a verified incident narrative and an evidence report without human help**. The goal was decomposed into six sub-goals, each with a measurable success criterion, so that progress could be checked at the end of every phase.

Table 3: Goal decomposition and success criteria

Goal	Sub-goal	Success criterion
G1	Ingest all logs into one event schema	Every non-blank line becomes exactly one event; zero silently dropped lines.
G2	Detect the attack chain	All six stages found; recall and precision against the answer key; decoy kept out of the chain.
G3	Explain the chain in plain English	Sentences cite event IDs and restate only established facts.
G4	Verify every sentence	No unsupported sentence appears in the final narrative; failures are repaired, dropped or replaced by facts.
G5	Present the result	One offline HTML report with timeline, clickable evidence, attempts-only and could-not-prove sections.
G6	Measure honestly	Evaluation on new cases against a baseline, a held-out hallucination test and an audit, with limits stated.

2.2 Requirements

Table 4: Functional and non-functional requirements

Type	Requirement
Functional	Parse three log formats; correlate across hosts; derive entities from data; build facts; narrate with a local LLM; verify; repair; render a report; run one command end to end.
Functional	Work without an LLM (`--no-llm`) so that a deterministic report is always available.
Non-functional	Free and local; deterministic where possible (seeded, sorted, byte-identical reruns); every statement traceable to a file and line number.
Non-functional	Safe handling of hostile text: log lines are attacker-controlled, so the report must never execute them as HTML or script.
Non-functional	No overfitting: analysis code never reads the answer key and never hard-codes case values.

2.3 Technology and tool selection

Table 5: Tools and technologies

Tool	Role	Reason for choice
Python 3.12	Implementation language	Standard library is enough for parsing and rules; few dependencies.
requests, jinja2, pytest	HTTP client, report template, tests	Only three third-party packages are needed.
Ollama	Local model server	Free, offline, reproducible options (temperature, seed, context size).
qwen2.5:7b-instruct	Narrator agent	Follows strict JSON formats well and answered fastest in the setup check.
llama3.1:8b	Judge agent	A different model family from the narrator, so the judge is independent of it.

Tool	Role	Reason for choice
git and GitHub	Version control	One commit per phase; the repository serves as the audit trail.
Claude Code	AI coding assistant	Implemented each phase from written specifications and prompts prepared by the team; all outputs were reviewed and tested.
HTML, CSS, JavaScript	Report format	A single offline file that can be e-mailed or printed.

Two different models are used on purpose. If the same model wrote a sentence and then judged it, it would tend to accept its own mistakes. The system warns loudly and records `judge_independent = false` whenever both roles are given the same model.

2.4 Dataset planning

Real incident logs rarely come with a trustworthy description of what actually happened, and sharing them raises privacy problems. A seeded **synthetic generator** was therefore planned. It writes the three log files, a host inventory and a hidden answer key that records the exact file and line number of every attack line. Because the key is hidden from the analysis code, it can be used to score the system objectively.

Table 6: Generated cases used in the project

Case	Difficulty	Seed	Log lines (auth / nginx / Windows)	Purpose
case01	Easy	42	265 / 426 / 96 (787)	Development; low noise, all evidence present.
case02	Hard	7	1,111 / 2,338 / 458 (3,907)	Development and demonstration; about five times the noise.
eval_e1, eval_e2	Easy	101, 102	generated at evaluation time	Evaluation only (new seeds never used before).
eval_h1, eval_h2	Hard	201, 202	generated at evaluation time	Evaluation only (new seeds never used before).

The hard difficulty adds a **decoy address** that attacks but never gets in, interleaved benign events, a **+90 second clock skew** on the Windows host, and **one deliberately deleted evidence line** (the creation of a backdoor account). These three traps test whether the system avoids false alarms, copes with unsynchronised clocks and admits what it cannot prove.

2.5 Work plan and timeline

The work was split into phases. After each phase the results were reviewed before the next phase was specified, and each phase ended with a git commit.

Table 7: Phase plan and outputs

Phase	Work	Main output	Commit
0	Setup	Folder structure, virtual environment, model check, project log (CLAUDE.md)	eb435ca
1	Synthetic generator	Three log types, planted attack, answer key, hosts.json	f49f848
2	Parsers and schema	One event schema, deterministic event IDs, clock-offset option	13767ac
3	Correlation rules	Six rules, chain builder, findings.json and chain.json	c138335

Phase	Work	Main output	Commit
4	LLM narrator and baseline	Cited narrative per stage; naive baseline on raw events	27c9a31
5	Verifier and repair loop	Structure, entity, semantic and judge layers; repair rounds	97d5cf7
5b	Hardening	Fact-sheet narration, quote-checked double judge, run metadata	0c02a0f
6a	Analyze command and report	One-command pipeline, offline HTML report, README and demo kit	47dd7d5
6b	Evaluation	Held-out hallucination pool, harness, audit kit, RESULTS.md	bd45452, 8a0d702
6b+	Audit in results	LLM-labelled audit added to the results, clearly marked	7b806fc

2.6 Design principles

- **Rules decide, the model explains.** The attack chain is found by deterministic rules. The language model never decides what the attack is; it only rewrites established facts.
- **Never trust a model to grade itself.** Final verdicts are derived in code from quoted evidence, and a second model that differs from the narrator does the judging.
- **No overfitting.** The analysis code never reads the answer key (a static test checks this). After the evaluation cases were generated, a freeze rule forbade changing any rule, prompt or threshold to improve a number.
- **Be honest about absence.** Missing evidence is reported as unprovable, and unprovable facts are never invented to make a story complete.
- **Reproducibility.** Seeds, sorted outputs, recorded model digests and a code hash allow a run to be repeated and checked.

2.7 Risk plan

Table 8: Risks and planned mitigations

Risk	Mitigation
The model hallucinates an entity or claim	Four-layer verifier; sentences that fail are repaired, dropped or replaced by verbatim facts.
Rules are tuned to the generated data	Entities derived from events only; new seeds for evaluation; a frozen held-out hallucination pool; code-hash comparison at the end.
Local models are slow	Batched rounds to limit model swaps; `--reuse` to rebuild a report in seconds; `--no-llm` fallback.
Host clocks disagree	A 300 second tolerance for cross-host links; an explicit offset flag; skew is never guessed.
A model request hangs	A 600 second timeout on every request, and a resumable evaluation harness.
Hostile text in logs reaches the report	Data embedded as escaped JSON and rendered with textContent; URLs shown as plain text; an injection test.

3. KEY COMPONENTS OF THE IDENTIFIED SKILL SETS

3.1 System architecture

Figure 1 shows the complete pipeline. Raw logs enter on the left of the diagram and a verified report leaves at the bottom. Blue boxes are rule-based components, orange boxes are LLM-driven agents, the green box is the verification stage, and grey boxes are data. The dashed path shows that the answer key is read only by the evaluation harness, never by the pipeline.

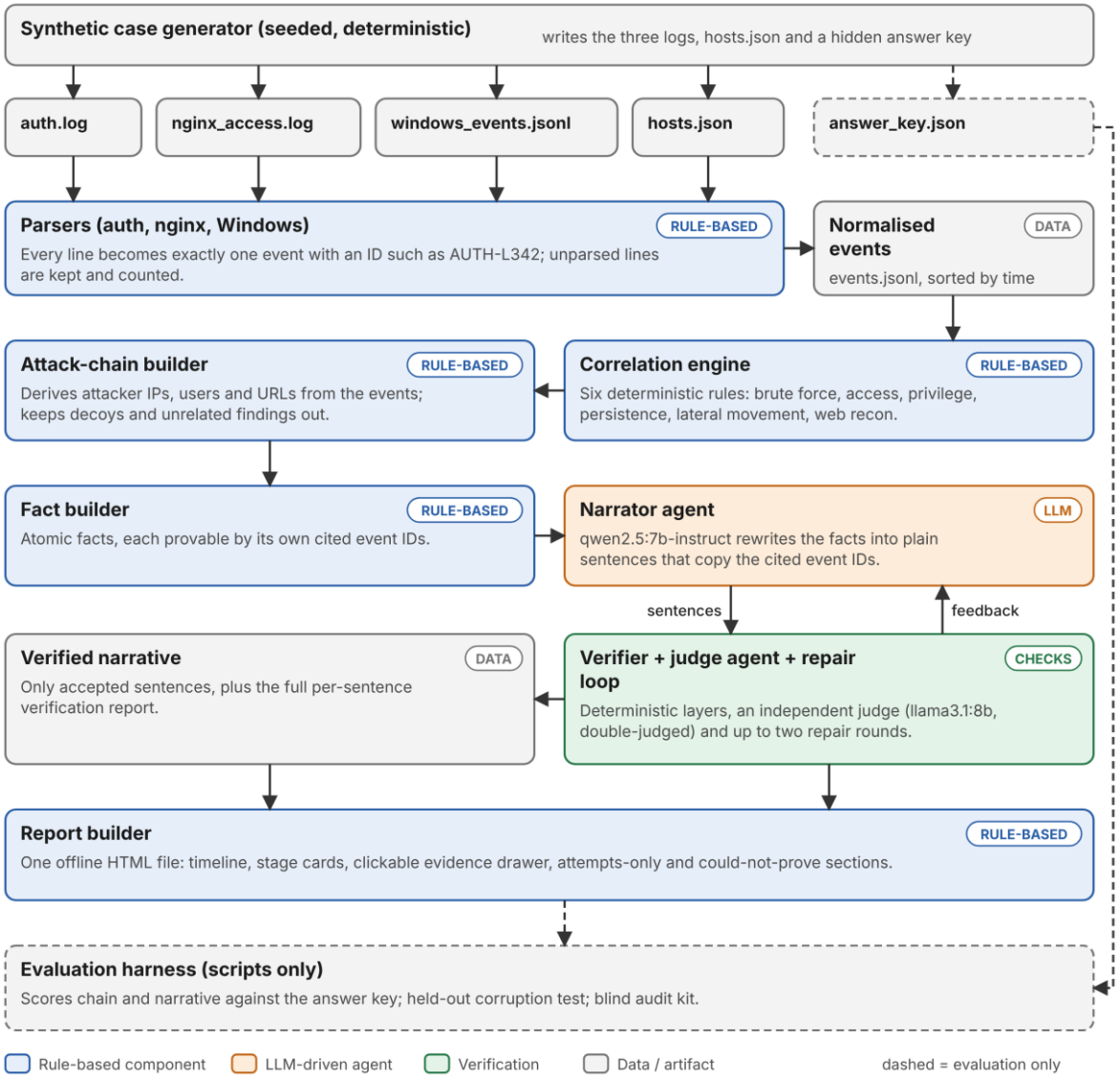


Figure 1: End-to-end architecture of EvidenceWeave

3.2 Agent roles and responsibilities

The pipeline is organised as eight cooperating agents. Each has a single responsibility, a defined input and output, and a contract with its neighbours. Six of them are deterministic programs; only the Narrator and the Judge call a language model.

Table 9: The eight agents

No.	Agent	Type	Responsibility	Input → Output
1	Ingest	Rule-based	Parse the three log formats into one schema; never drop a line; keep unparsed lines and count them.	raw logs, hosts.json → events.jsonl
2	Correlation	Rule-based	Apply six rules; build the attack chain from shared entities; separate decoys and unlinked findings.	events → findings.json, chain.json
3	Fact	Rule-based	Turn each stage into atomic facts, each provable from its own cited events.	chain, events → facts per stage
4	Narrator	LLM (qwen2.5:7b)	Rewrite facts into plain sentences that copy fact IDs and cite event IDs.	facts → sentences (JSON)
5	Verifier	Rule-based	Run structure, entity and semantic checks; derive the verdict in code.	sentences → verdicts
6	Judge	LLM (llama3.1:8b)	Split each sentence into claims and quote one exact log line per claim; run twice.	sentence + cited lines → claims, quotes
7	Repair controller	Control logic	Send failed sentences back with reasons (at most two rounds); choose the best attempt; fall back to facts.	failures → feedback, final narrative
8	Report	Rule-based	Build the offline HTML report with evidence index.	verified narrative, facts → report.html

3.3 Multi-agent orchestration

Figure 2 shows how the agents are coordinated. A deterministic **pipeline controller** (the `analyze` command) invokes the agents in a fixed order and enforces the limits. The agents never call one another directly. Each reads files written by the previous agent and writes its own files to a shared artifact store (`output/<case>/`), which makes every hand-over inspectable and every step repeatable.

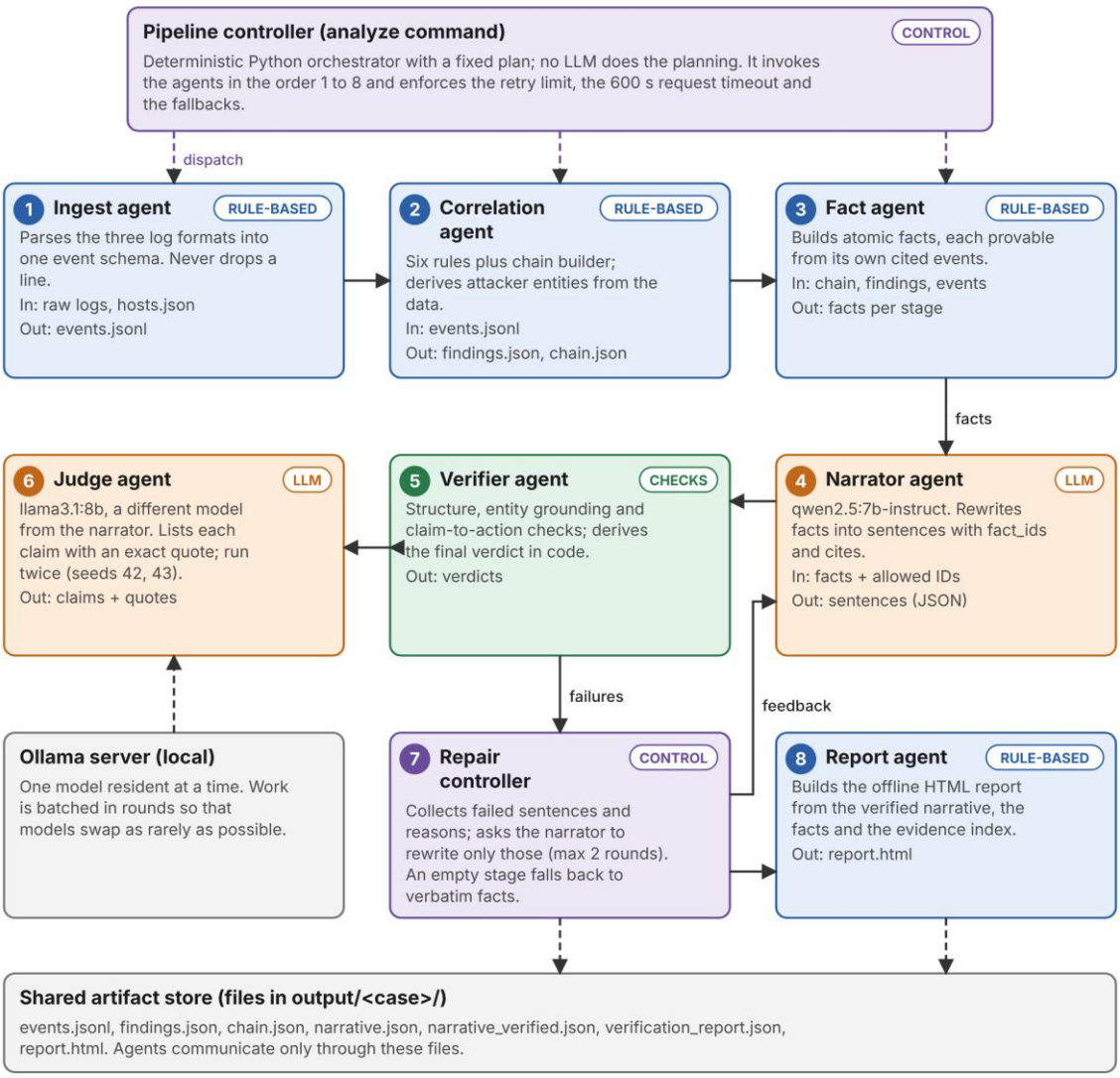


Figure 2: Agent orchestration: controller, eight agents, and the shared artifact store

The numbers on the cards give the invocation order. Agents 1 to 3 run once. Agents 4, 5 and 6 form the generate-and-check loop, and agent 7 closes the loop by sending failures back to agent 4. Agent 8 runs last. The Ollama server in the lower left holds only one model in memory at a time, which drives the batching rule described below.

Table 10: Orchestration design decisions

Decision	What was chosen	Why
Planning	A fixed plan executed by a deterministic controller; no LLM planner.	Forensic work must be repeatable. A model choosing its own steps would make runs differ and be hard to audit.
Communication	Agents exchange files in a shared artifact store.	Every hand-over can be opened and checked; stages can be re-run or skipped (`--reuse`).
Coordination of models	Batched rounds: all narrator calls of a round come before that round's judge calls.	Ollama swaps models in memory; batching limits swaps to at most one per round.
Independence	Narrator and judge are different models.	A model tends to approve its own mistakes.
Termination	At most two repair rounds; the best attempt per stage is kept.	Guarantees the loop ends; progress is never lost because accepted sentences are kept.

Decision	What was chosen	Why
Failure handling	One retry for unparseable JSON; 600 s request timeout; a stage with no accepted sentence falls back to verbatim facts; `--no-llm` mode.	The pipeline always produces a usable, honest report.

A note on the word "agent"

Only the Narrator and the Judge use a language model. The other components are called agents because each has one responsibility, its own input and output, and a defined contract with the rest of the system. The orchestration is a fixed controller, not an autonomous planner. This was a deliberate choice for reliability and is discussed again under limitations.

The repair loop in detail

Figure 3 shows one full verification cycle as a sequence diagram. In round 0 the narrator writes the first draft at temperature 0. The verifier applies the deterministic layers and sends only the survivors to the judge, which runs twice. In each repair round the controller returns only the failed sentences, with failure codes written in plain words, to the narrator. Repair calls use temperature 0.3 and a new seed so that the model can actually change its answer; accepted sentences from earlier attempts are kept unchanged.

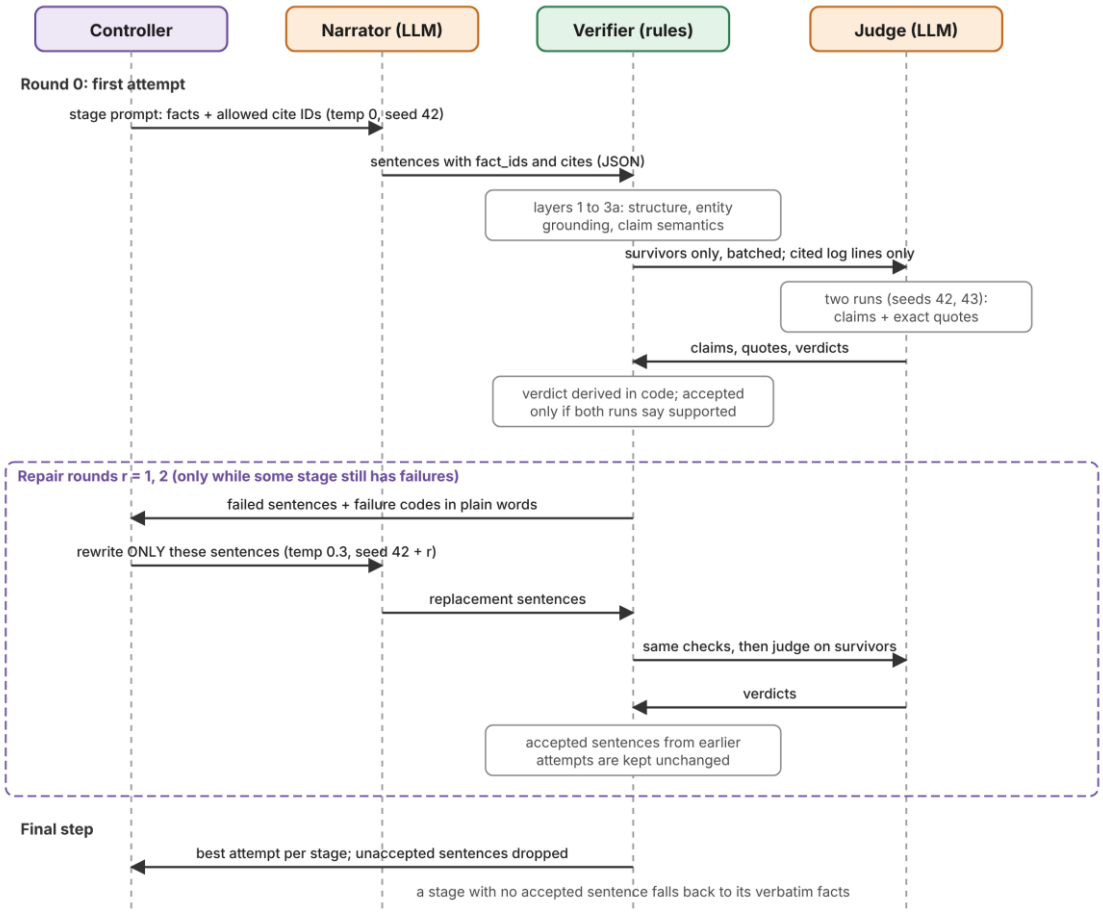


Figure 3: Sequence of one verification and repair cycle

3.4 Synthetic data generation and attack storyline

The generator plants the same six-stage attack, carried out by one attacker address, into every case and surrounds it with realistic benign activity: normal SSH sessions, administrator sudo use, scheduled jobs, web traffic from many addresses and ordinary Windows logons. Figure 4 shows the hard case as it appears in the logs.

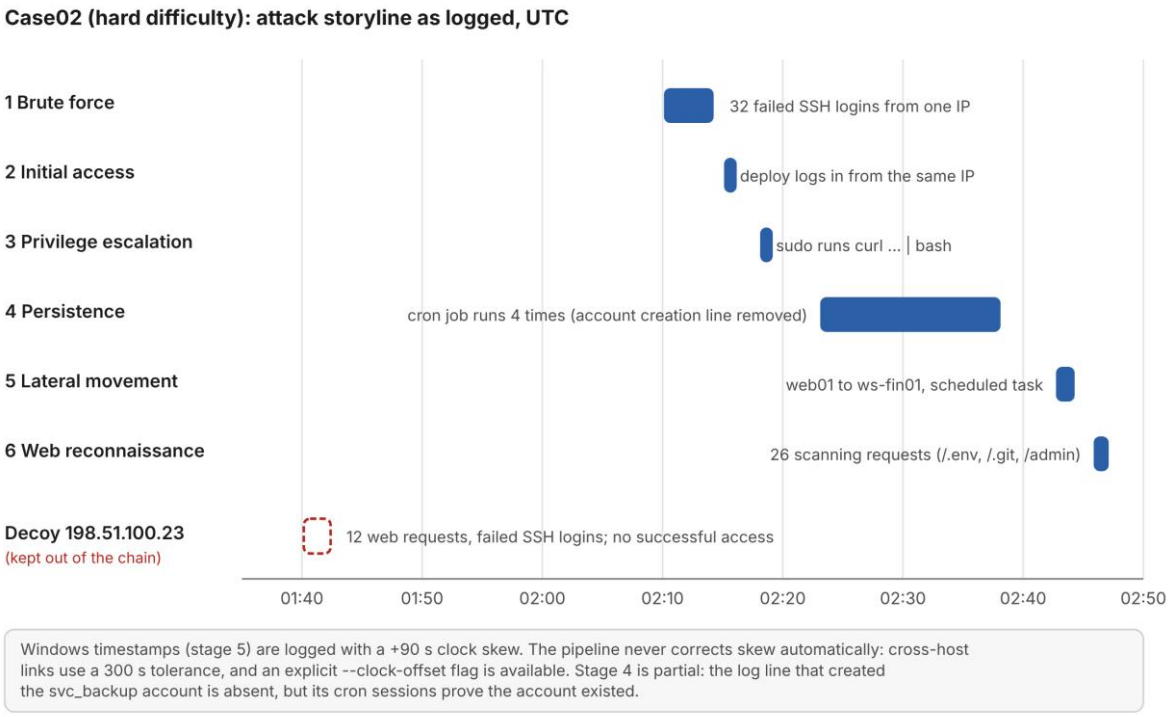


Figure 4: Attack storyline of case02 as logged (UTC), with the decoy lane

Table 11: Attack stages in case02

Stage	Name	Logged time (UTC)	Evidence lines	What happens
1	Brute force	02:10:07 – 02:14:15	32	Many failed SSH logins from one external address, mixing invalid names and the valid user "deploy".
2	Initial access	02:15:07	1	A successful SSH login as deploy from the same address.
3	Privilege escalation	02:18:07	1	deploy runs sudo with a download-and-run command (curl piped to bash).
4	Persistence	02:23:07 – 02:38:07	12	A new account svc_backup runs a cron job four times. The line that created the account was deleted from the log.
5	Lateral movement	02:42:44 – 02:44:17	7	From web01 (10.0.0.5): failed and then successful network logon on ws-fin01, special privileges, encoded PowerShell, scheduled task.
6	Web reconnaissance	02:45:52 – 02:47:07	26	The attacker address scans for /.env, /.git/config, /admin and similar paths.

The Windows host clock runs 90 seconds ahead, so stage 5 is logged at 02:42:44 although it happened at 02:41:14. The decoy 198.51.100.23 produces failed SSH logins and web scans but never logs in.

3.5 Event normalisation (Ingest agent)

Each parser turns one log line into one **event**. The identifier is built from the source and the line number, so that any sentence can be traced to the exact raw line: `AUTH-L342` is line 342 of the auth log, `NGX-L560` is line 560 of the nginx log and `WIN-L88` is line 88 of the Windows log. Nothing is dropped silently: a line that cannot be parsed becomes an event with action `unparsed`, and blank lines are counted separately. In both generated development cases every line parsed, so the unparsed count was zero.

Table 12: Normalised event schema

Field	Meaning
event_id	Deterministic ID from source and line number (AUTH-L342, NGX-L560, WIN-L88).
timestamp / raw_timestamp	Corrected UTC time used for sorting, and the time exactly as logged.
clock_corrected	True only if an explicit clock offset was applied to this host.
host, user, src_ip, dst_ip	Where the event happened, who did it and from which address (or none).
source_type, action	auth, nginx or windows; and one of 18 fixed actions such as ssh_login_failed, ssh_login_success, sudo_command, user_created, cron_command, web_request, win_logon_success, win_process_created, win_task_created.
raw_file, raw_line, raw_text	The original line, unmodified, with its file and line number.
extra	Source-specific details such as the command, status code or logon type.

Clock-skew policy. Skew is never corrected automatically, because no reliable anchor event exists to measure it. An analyst may pass `--clock-offset HOST=SECONDS`; otherwise the raw time is used and cross-host links tolerate a 300 second difference.

3.6 Correlation agent and attack chain

Six independent rules each take the events and return findings. Thresholds are configurable and were fixed before the evaluation cases were generated.

Table 13: Correlation rules and default thresholds

Rule	Fires when	Default
Brute force	Many failed logins from one address within a time window.	8 failures in 600 s
Initial access	A successful login from an address whose brute-force burst ended shortly before.	within 1,800 s
Privilege escalation	A sudo command matching a suspicious pattern (download piped to a shell, encoded payloads, reverse shells) by the compromised user.	pattern list
Persistence	A user created after access, a cron command with a suspicious pattern or URL, or a scheduled task. If the cron user has no creation event, a missing-evidence finding is added.	pattern list
Lateral movement	A network logon from the address of an inventory host that is already compromised, followed by special privileges, a suspicious process and a scheduled task on the target.	900 s window
Web reconnaissance	Many requests with a high share of 403/404 answers, or several distinct sensitive paths.	15 requests, 60% errors, 900 s; or 3 sensitive paths

The **chain builder** then links findings. It starts from the addresses that both brute-forced and logged in, learns the compromised user, and grows the attacker set through shared entities: payload URLs and the addresses inside them, and the pivot host recorded in the host inventory. A finding joins the chain

only if it shares an entity with that set and fits the time order. Figure 5 shows the result for case02, including what was deliberately left out.

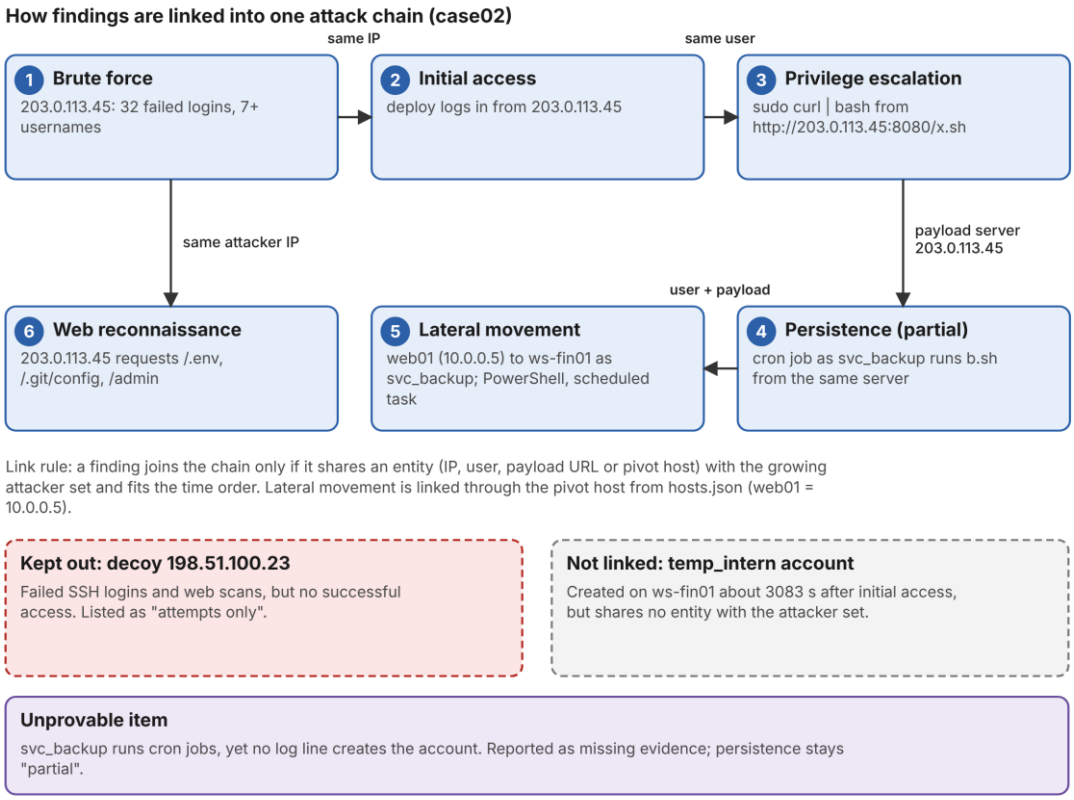


Figure 5: Attack chain of case02: how findings link, and what stays out

Anti-overfitting rules
The analysis code never reads the answer key or the attack information in the manifest, and never contains the attacker address, user names or payload URL of the generated cases. A test copies a case, deletes the key and checks that the chain is identical. A static test checks that no module of the package contains the text "answer_key". Rules were tested on three further seeds in both difficulties, and none was changed afterwards to improve a result.

3.7 Fact builder

For every stage the fact builder produces up to eight **atomic facts** in a fixed priority order: time window, main actor, key action, counts, links, and status. Each fact is a deterministic sentence that makes exactly one checkable claim and cites the events that prove it by themselves. For example, the privilege-escalation stage contains the fact that the user ran a sudo command on web01 and quotes the command exactly as logged, citing `AUTH-L330`. Facts about failed attempts cite failure events, facts about a success cite the success event, and a partial stage carries a fact stating what could not be shown. Every fact must itself pass the deterministic verification layers; a fact that fails is treated as a bug in the builder.

3.8 Narrator agent

The narrator is `qwen2.5:7b-instruct`, called through Ollama with temperature 0, seed 42 and a 16,384-token context. It receives the numbered facts of one stage together with the list of allowed cite IDs, and must return JSON of the form `{"sentences": [{"text", "fact_ids", "cites"}]}`. Its instructions forbid adding any information, number, time or entity that is not in the facts. One call is made per stage plus

one for the executive summary, because small prompts work far better with a 7B model than one large prompt.

An earlier design, **raw mode**, gave the model the stage evidence lines and let it describe them. A diagnosis of 47 failing sentences showed that 33 failed because the prompt invited claims that no allowed citation could support. This led to the fact-sheet design above, which is now the default. Raw mode is kept so that the two approaches can be compared.

3.9 Verifier and independent judge

Every sentence passes through up to four layers (Figure 6). The deterministic layers are cheap and catch most errors; the judge handles what rules cannot see. A sentence is accepted only if it passes every layer.

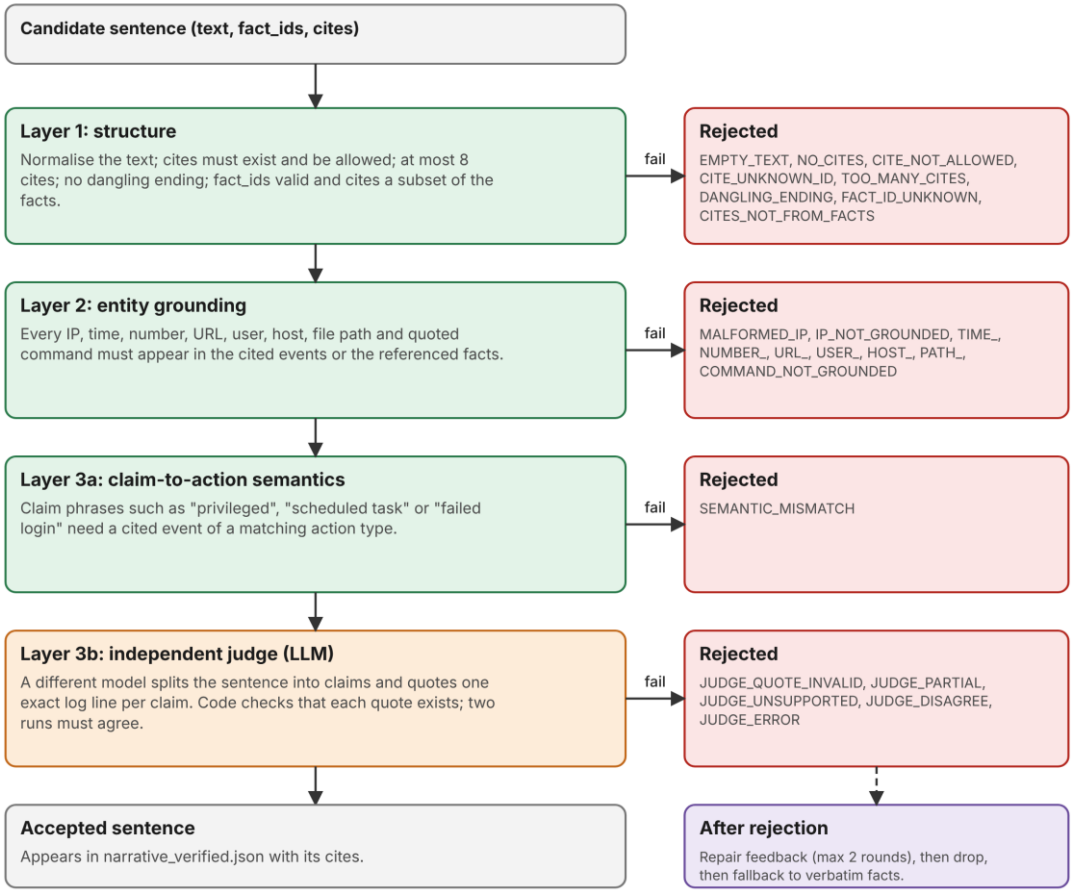


Figure 6: The four verification layers and their failure codes

Table 14: Verification layers

Layer	What it checks	Example of an error it catches
1 Structure	Cites exist and are allowed; at most 8 cites; fact IDs valid; cites are a subset of the facts; no dangling ending; junk characters normalised.	A sentence that ends in "as shown by."
2 Entity grounding	Every IP, time, number, URL, user, host, file path and quoted command must appear in the cited events or the referenced facts.	An address with a dropped dot, such as 203.0113.45; an invented path.
3a Semantics	Claim phrases need a cited event of a matching action type (a table that is easy to extend).	Calling a plain logon a "privileged logon".

Layer	What it checks	Example of an error it catches
3b Judge	A second model lists the claims and quotes one exact log line for each; code checks that every quote exists in the cited lines; two runs (seeds 42 and 43) must both say supported.	A claim with no checkable entity, such as "the attacker wiped the history".

The final verdict is **derived in code**, not taken from the judge's own answer. If the judge says a claim is supported but its quote does not occur in the cited lines, the sentence is rejected with `JUDGE_QUOTE_INVALID`. Acceptance is conservative: partial, unsupported, disagreeing and errored judgements all lead to rejection.

3.10 Report builder

The report agent produces one self-contained HTML file with no external fonts, scripts or network requests, so it works offline and can be e-mailed. It contains a header verdict built only from chain counts, a timeline with a separate lane for attempts-only actors, an executive summary, one card per stage showing **established facts** and **verified sentences** side by side, a drawer that shows the raw log line, file name, line number and neighbouring lines when an event ID is clicked, an attempts-only section, a "what could not be proven" section, and a method-and-limits panel.

Because log lines are attacker-controlled, all log text is embedded as escaped JSON and displayed with `textContent`; URLs from logs are shown as plain text; and a test injects script and HTML strings into every field to confirm that none becomes executable. Re-running with the same inputs and the same timestamp produces a byte-identical file.

3.11 Implementation summary

```
evidenceweave/
  generator/    synthetic cases, answer key, host inventory
  parsers/      auth, nginx, Windows parsers; pipeline
  correlation/  six rules, findings, chain builder, configuration
  narrator/     facts, prompts, Ollama client, narrate, baseline
  verifier/     structure, entities, semantics, judge, repair, corruption tests
  report/       HTML report builder
scripts/        check_ollama, check_against_key, evaluate_cases, audit tools
tests/          212 automated tests (no test calls a real model)
docs/           README, demo script, RESULTS.md, sample reports
```

Table 15: Command-line interface

Command	Purpose
`generate`	Create a synthetic case with a seed and a difficulty.
`parse`	Normalise the logs into events.jsonl (optional `--clock-offset`).
`correlate`	Run the rules and the chain builder; write findings.json and chain.json.
`narrate`	Write the narrative in facts or raw mode, or the naive baseline.
`verify`	Verify and repair the narrative; run the baseline and corruption tests.
`analyze`	Run the whole pipeline in one command and open the report; `--no-llm` and `--reuse` available.
`report`	Rebuild the HTML report from existing outputs without any model call.
`evaluate`	Run the evaluation harness (delegates to the scripts folder).

Every output file records run metadata: the Ollama version, model digests, all model options, prompt versions, the git commit and the date. Tests use a fake language model, so the suite runs in seconds without Ollama.

3.12 Mapping to the evaluation parameters

Table 16: Where each evaluation parameter is addressed

Evaluation parameter	Where addressed
Agent Planning and Goal Formulation	Section 2.1 (goal decomposition), 2.6 (design principles), Section 3.3 (fixed plan and its rationale).
Agent Architecture and Integration	Section 3.1 (architecture), 3.2 (agent roles), 3.5 to 3.10 (components and their contracts).
Multi-Agent Orchestration	Section 3.3 (controller, artifact store, batched rounds, repair loop, termination and failure handling).
Implementation and Functionality	Section 3.11 (implementation, commands), Section 4.4 (working report).
Testing and Performance Evaluation	Section 4.1 to 4.3 (tests, evaluation design and results).
Innovation	Rules decide and the model explains; fact-sheet narration; quote-checked double judge; held-out hallucination pool; honest missing-evidence handling.
Presentation and Viva	Section 5 (presentation outline, demo script, questions).
Report Preparation	This report.

4. PROOFS OF OUTCOME AND FEEDBACK

4.1 Testing

The project has 212 automated tests. None of them calls a real language model; a scripted fake model stands in for both the narrator and the judge, so every test is fast and repeatable. The number of passing tests grew with every phase.

Table 17: Passing tests after each phase

Phase	0–1	2	3	4	5	5b	6a	6b
Tests	13	37	68	95	140	169	190	212

Table 18: What the tests cover

Area	Examples of what is tested
Generator	Same seed gives byte-identical files; every answer-key line number really contains its marker; hard case has a missing line that truly is absent.
Parsers	Every line becomes exactly one event; garbage and truncated lines never raise; clock offset changes only the corrected time.
Correlation	Each rule fires on a positive and not on a near-miss; benign mistyped passwords do not trigger; recall of at least 0.9 on three extra seeds in both difficulties; the analysis never reads the key.
Narrator	Evidence caps, retry on bad JSON, citation problems left untouched for the verifier, template sections not produced by the model.
Verifier	Each layer and failure code; the judge prompt contains only the sentence and its cited lines; batched call order; fallback behaviour; corruption generators.
Report	All sections present; no external resources; injection strings never become markup; every cite chip resolves to an evidence entry; byte-identical rebuilds.
Evaluation	Strict versus loose coverage, entity matching, Wilson intervals, resumable stages, audit sampler and scorer.

4.2 Evaluation design

Four new cases (two easy, two hard, seeds 101, 102, 201 and 202) were generated after the code was frozen. The following arms and measures were used.

Table 19: Arms and metrics

Item	Definition
Chain detection	Evidence lines found by the rules compared with the answer key: recall, precision, decoy and benign-looking lines kept out, missing evidence detected.
Chain arm	Rules and chain, facts-mode narrator, four-layer verifier with repair (narrator qwen2.5:7b-instruct, judge llama3.1:8b).
Baseline arm	The same narrator model given the raw sorted events in chunks of 120, with no rules and no chain, asked to write a cited incident narrative.
Strict step coverage	A step counts as covered if a final sentence with at most 8 cites cites at least one of that step’s key evidence lines. Loose coverage removes the cite limit.
Entity coverage	Share of attacker entities (address, user, payload host, pivot host) that appear as exact tokens in the final text.
Held-out corruption test	Clean accepted sentences are corrupted with 14 invented claims without entities and 10 invented entities that the verifier’s authors never saw; the share rejected is measured, with clean controls for false rejection.

Item	Definition
Role swap	Narrator and judge models exchanged on two cases to check that results do not depend on one pairing.
Audit	A sample of 32 sentences labelled as supported, partly supported or unsupported using only the cited log lines.

4.3 Results

Chain detection

Table 20: Chain detection on the four evaluation cases

Measure	Result
Key evidence lines found	329 of 329
Extra lines not in the key	0
Recall and precision	100% and 100% on every case
Decoy address and benign-looking lines in the chain	None
Missing evidence (hard cases)	Detected and reported as unprovable

Perfect detection must be read with care: the cases come from the same generator that the rules were written against, so this shows that the rules work across the generator's variations, not that they work on real logs.

Chain narrative versus baseline

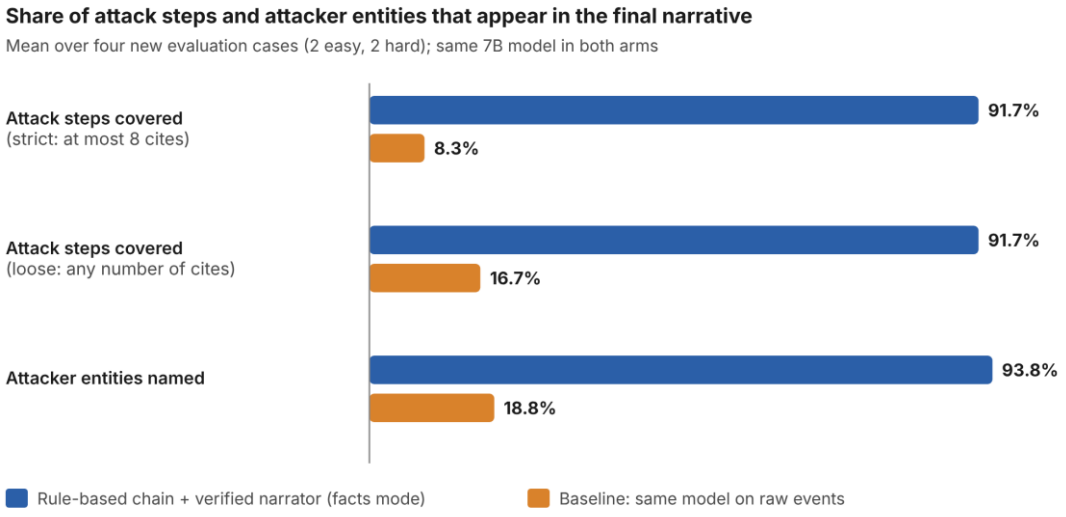


Figure 7: Attack steps and attacker entities present in the final narrative (mean of four cases)

With the same 7B model, the baseline lost most of the attack: strict step coverage was 8.3% against 91.7% for the chain arm, and none of the baseline's accepted sentences covered a step. Even when the cite limit was removed, the baseline reached only 16.7%. The baseline also labelled routine cron jobs and internal logins as threats. This is the main justification for letting rules find the attack.

Efficiency

Table 21: Cost of the approaches

Run	Time	Tokens	Model calls
Chain narrator, raw mode, case01 (Phase 4)	48.5 s	9,919	7
Baseline on raw events, case01 (Phase 4)	212.1 s	100,824	8
Full analyze with verification, case02 (live)	1,009 s (narrate 74.6 s, verify and repair 934.2 s)	not recorded	many
analyze with `--no-llm`, case02	about 0.3 s	0	0

Without verification the chain narrator is about four times faster than the baseline and uses about ten times fewer tokens. Verification with the double judge and repair is the expensive part: a live run takes about 17 minutes. This is why the demonstration uses a saved report or the `--no-llm` mode.

Verification behaviour

Table 22: Raw-mode versus fact-sheet narration (Phase 5b, development cases)

Case	Measure	Raw mode	Facts mode
case01	Accepted sentences	2	14.7 (range 14–15, 3 runs)
case01	Fallback units	5	0
case02	Accepted sentences	6	15 (3 runs, identical)
case02	Fallback units	1	0

In the live case02 run, 27 sentences were generated. Eight were accepted at the first attempt, and repair added four and then three more, giving 15 accepted and 12 dropped. Judge self-agreement was 0.80. The higher acceptance in facts mode partly reflects an easier task, because the model restates pre-verified facts, and should not be read as a smarter model.

Hallucination test on a held-out pool

Table 23: Held-out corruption test (pooled over two cases)

Measure	Result
Corrupted sentences rejected	119 of 120 (99.2%, 95% confidence interval 95.4–99.9%)
Clean control sentences wrongly rejected	2 of 28 (7.1%)
Corrupted sentence that was accepted	"The attacker wiped the shell history afterwards." appended to a clean sentence (no checkable entity)

Every corruption that contained a checkable entity (a changed address, time or number, a non-existent or foreign cite, an invented path) was caught by the deterministic layers. The one miss is a claim with no entity, which only the judge can reject, and it did not. On the earlier development pool the corresponding results were 25 of 27 in Phase 5 and catch rates of 0.957 and 1.000 on the two cases in Phase 5b.

Role swap

Swapping the narrator and judge models on the other two cases gave similar numbers, and the judge stayed independent in both runs. On eval_h2 the swapped pairing was worse, covering 5 of 6 steps and 3 of 4 entities against 6 of 6 and 4 of 4 with the default pairing, so the choice of pairing does matter.

Audit of 32 sentences

> **Important notice**
> The 32 audit labels were produced by an LLM from the same model family as the narrator. No human labelled them, so they are **not independent ground truth** and the results are indicative only. A human audit remains to be done.

Table 24: Audit labels versus the verifier decision

Verifier decision	Supported (S)	Partly supported (P)	Unsupported (U)	Total
Accepted	20	4	0	24
Rejected	5	2	1	8

Of the 24 accepted sentences, 20 were labelled supported (83.3%, interval 64.1–93.3%) and all 24 were labelled supported or partly supported (100%, interval 86.2–100%); none was labelled unsupported. The four accepted sentences labelled "partly supported" overstate what their cited lines show: three claim that failed attempts "ended" at a time, which one line cannot prove, and one calls events "related to persistence", which is an interpretation. Of the 8 rejected sentences, 5 were labelled supported, so some rejections look unjustified; the sample is too small (interval 30.6–86.3%) to say how often.

4.4 Proof of outcome: the generated report

Figures 8 and 9 show pages of the report that EvidenceWeave produced for case02 in the live run. The header verdict reads: "Compromise confirmed: 5 of 6 stages established, 1 stage partially unprovable; 1 actor with attempts only and no successful access." The timeline places the decoy in its own lane, and each stage card separates deterministic facts from verified sentences, with clickable event IDs.

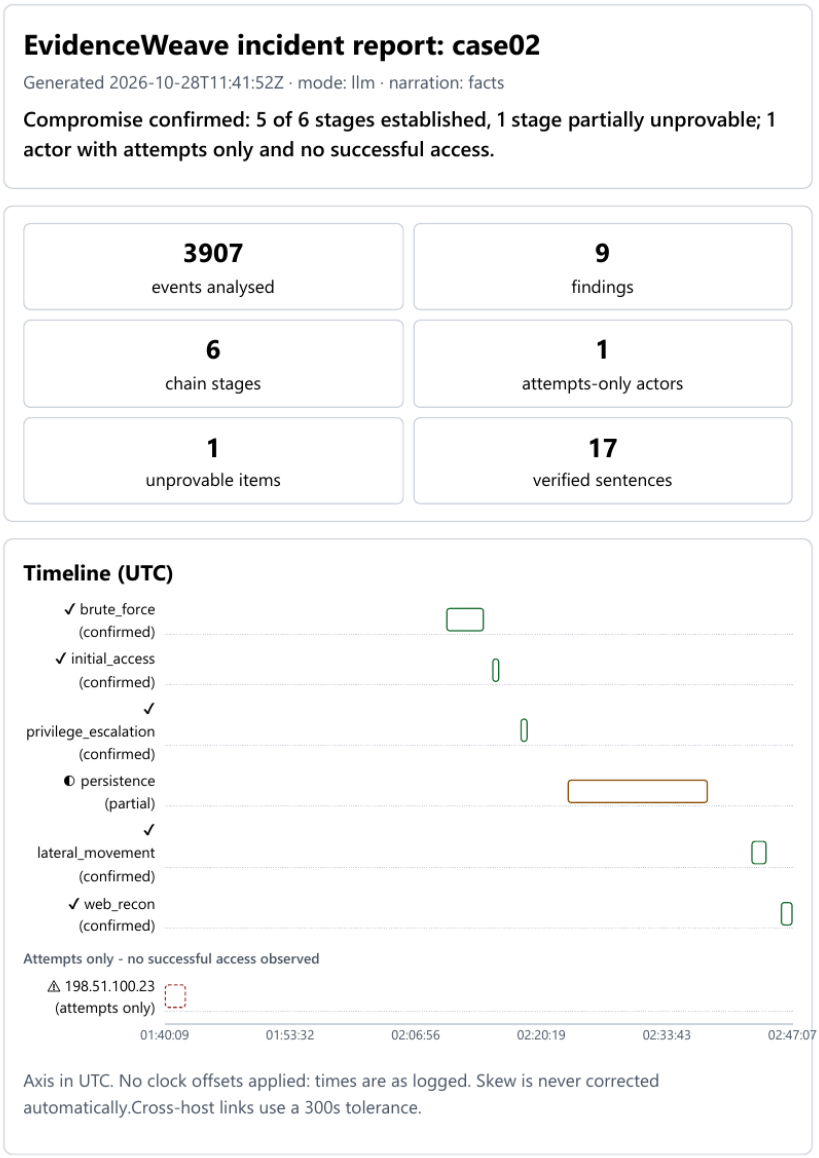


Figure 8: Report header, summary tiles and timeline for case02

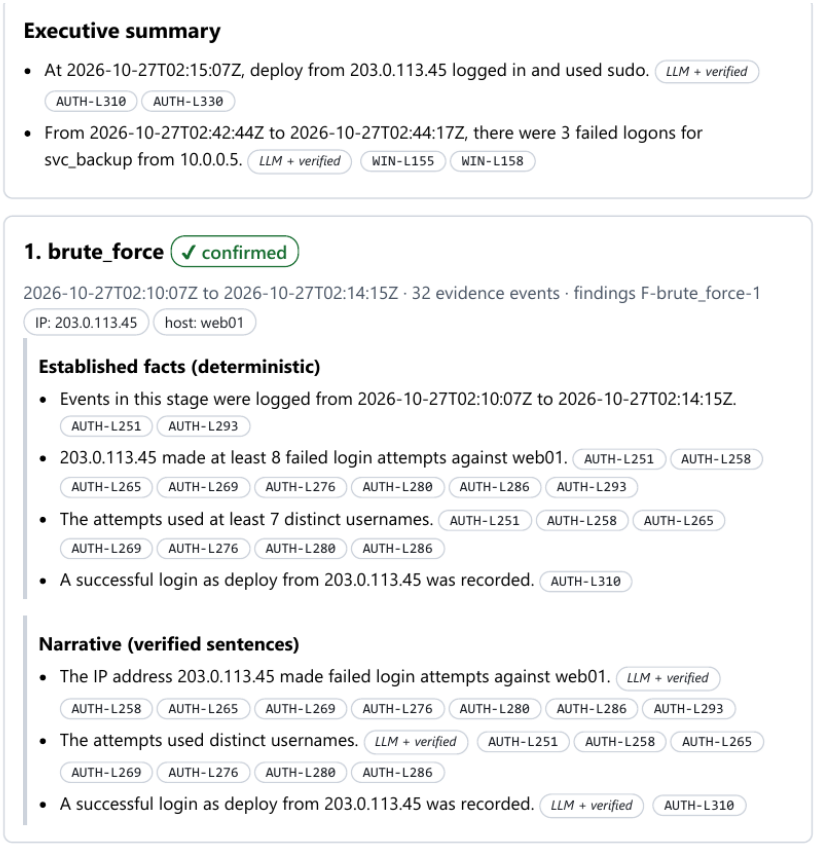


Figure 9: Executive summary and the brute-force stage card with facts, verified sentences and cite chips

Other sections of the same report: the "attempts only" section states that no successful access from 198.51.100.23 was observed; the "what could not be proven" section states that account svc_backup runs cron jobs but no event creating it appears in the logs; and the "not linked" list explains that the temp_intern account shares no entity with the attacker set.

Defects in model output caught by the verifier

Table 25: Defects seen in the first model output and the layer that handled them

Defect	Handled by
An address written as 203.0113.45 (a dot dropped)	Layer 2 (MALFORMED_IP); repaired in round 1.
A sentence ending in stray quote characters	Layer 1 normalisation.
A plain logon described as a "successful privileged logon"	Layer 3a (SEMANTIC_MISMATCH); sentence dropped because it was not fixed.
Event IDs written inside the sentence text	Layer 1 moved them into the cites.

4.5 Troubleshooting and issues resolved

Table 26: Issues found during the project and how they were fixed

Issue	Cause	Fix
Virtual environment had no Scripts folder	The Python on the PATH was an MSYS2 build (3.14).	Rebuilt the environment from standard Python 3.12.4.
Windows events appeared 90 s late	The Windows host clock is skewed.	300 s cross-host tolerance and an explicit `--clock-offset` flag; skew is never guessed.
Mismatched times between events and chain	events.jsonl held corrected times while chain.json held raw times.	The pipeline now stores its offsets in pipeline_meta.json.

Issue	Cause	Fix
Answer key missed four attack lines	Cron "session closed" lines were not tagged.	Key fixed; log files verified byte-identical.
Closing quote deleted from a command	Junk-quote clean-up removed a legitimate quote.	Balanced-quote rule with tests.
Sentence ending "as shown by."	Inline ID removal left a dangling phrase.	Dangling-ending check (DANGLING_ENDING).
Ordinary words treated as user names	The user-name extractor was too loose.	Suffix rule and a larger stop list.
Repair returned near-identical text	Temperature 0 with the same seed.	Repair at temperature 0.3, new seed, sentence-level feedback.
Most failures were unfixable	33 of 47 failures were prompt-design problems.	Fact-sheet narration became the default.
Evaluation stalled for about 57 minutes	A model request hung.	600 s request timeout (logged); resumable harness.
HTML template broke the report styles	A CSS rule collided with the template comment syntax.	Changed the comment delimiters.

4.6 Feedback and actions taken

Feedback came from three sources: the automated checks and diagnosis, the audit, and a review of the generated report. The table lists what was found and what was done. Under the freeze rule, nothing found after the evaluation cases were generated was fixed in the code; these items are documented as known limits instead.

Table 27: Feedback and resulting actions

Finding	Source	Action
Most Phase 5 failures came from the prompt, not the model	Diagnosis of 47 failures	Redesigned to fact-sheet narration before the evaluation.
The judge disagrees with itself in 4–20% of cases and its own verdict often differs from the code-derived one	Phase 5b live runs	Verdict derived in code from quoted lines; double judging.
Counts, absence claims and interpretations can be accepted although the cited lines do not fully show them	Audit	Documented as a known verifier gap; not tuned away.
One accepted corruption: a claim with no entity	Held-out test	Documented; the held-out pool was not edited afterwards.
A cut-off sentence ending in a backslash passed all deterministic layers (rejected only by the judge)	Audit	Recorded as an open issue; not fixed because of the freeze rule.
Header says 17 verified sentences while statistics say 15 accepted	Report review	Both are right: 15 model sentences plus 2 template sentences; to be relabelled.
The executive summary is thin and one sentence attaches the login time to the later sudo action	Report review	Documented; stage cards carry the full story.

Class teacher feedback

Space reserved for the evaluator’s comments after the presentation.

4.7 Limitations and threats to validity

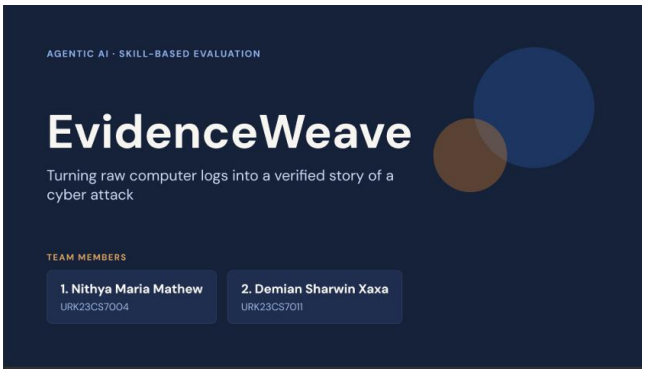
- All cases come from one synthetic generator. The results do not show performance on real logs.
- Sample sizes are small and each setting was run once, so differences are indicative. The judge is not perfectly reproducible even at temperature 0.
- The models are small (7B and 8B) and local; larger models may behave differently.
- The audit labels come from an LLM and are not independent ground truth.
- Fact-sheet narration is deliberately plain. The narrator mostly restates verified facts, so its higher acceptance rate partly reflects an easier task. Only about half of the facts appear in accepted sentences.
- Baseline sentences cite wide ranges of events, which inflates loose coverage and lowers strict coverage; both are reported.
- The held-out pool was frozen before the runs, but the verifier was designed knowing the original pool.
- Known verifier gaps: overstated sentences (counts, absence, interpretation), claims without a checkable entity, and a time attached to the wrong action in one sentence.
- Code hash: one file changed during the evaluation (the model request timeout was reduced from 1,800 s to 600 s). It changes no rule, prompt or threshold and is logged.
- Absence of evidence in logs is not proof of absence.

4.8 Conclusion and future work

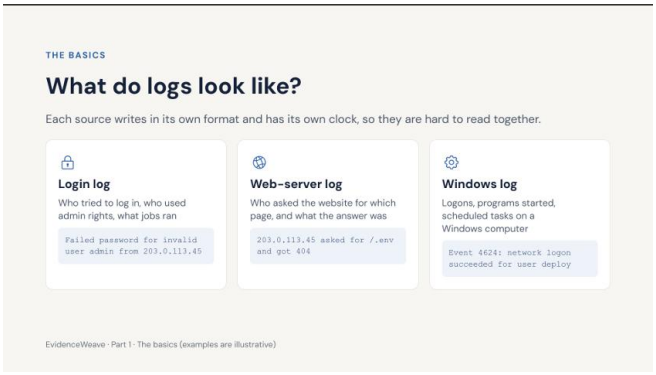
EvidenceWeave shows that a small local model can be useful in forensic reporting when it is constrained by the right architecture. The rules found all 329 attack evidence lines in the four evaluation cases with no false additions, kept the decoy and the unrelated account out of the chain, and reported the deleted evidence as unprovable. The verifier rejected 119 of 120 deliberately corrupted sentences, and the final narrative covered 91.7% of attack steps where the same model without the rules covered 8.3%. The costs are time, a plain narrative, and a judge that is not fully reliable.

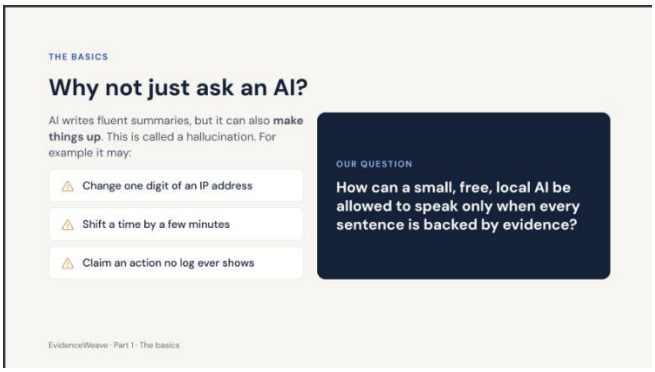
Future work: test on real or public log collections; add a human audit; strengthen checks for counts, absence claims and interpretations; improve the executive summary; estimate clock skew when a reliable anchor event exists; and compare larger models.

5. POWERPOINT PRESENTATION













THE ATTACK WE LOOK FOR

A six-stage break-in

Stage 1
Brute force
Guessing passwords again and again

Stage 2
Initial access
One guess works and the attacker gets in

Stage 3
Privilege escalation
Gets admin power on the machine

Stage 4
Persistence
Leaves a back door to return later

Stage 5
Lateral movement
Jumps to a second computer on the network

Stage 6
Web reconnaissance
Scans the website for secret files

THE SYSTEM

How it works in six steps

Step 1 **Read logs** — Turn every line into one common format

Step 2 **Find attacks** — Six rules spot suspicious patterns

Step 3 **Link the chain** — Connect findings that share the same attacker

Step 4 **Write facts** — Small, provable statements for each stage

Step 5 **AI explains** — A local model writes plain sentences

Step 6 **Check and report** — Verify, repair, then build the report

Rule-based, no AI · AI model · Verification

THE TEST DATA

Fake attacks with a hidden answer key

A generator writes realistic logs and plants the attack. It also writes an answer key that the analysis code can never read, so we can score the system honestly. No real personal data is used.

Easy case
787
log lines, little noise

Hard case
3,907
log lines, with three traps

Decoy attacker
Attacks but never gets in

Clock skew
Windows clock runs 90 seconds fast

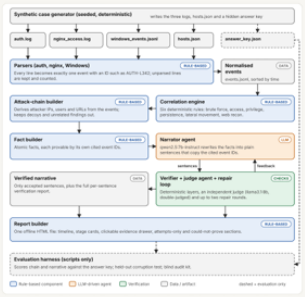
Deleted evidence
One log line was removed

THE SYSTEM

The full architecture

Raw logs go in at the top. A verified report comes out at the bottom.

Blue boxes: rule-based code
Orange boxes: the AI agents
Green box: verification
Dashed path: the answer key is read only by the scoring scripts, never by the pipeline



STEP 1 · READ LOGS

Every line becomes one event

RAW LINE 342 OF THE LOGIN LOG

```
Oct 27 02:10:07 web01 sshd: Failed
password for invalid user admin
from 203.0.113.45
```

ONE COMMON EVENT
event_id: AUTH-L342
host: web01 · **user:** admin
src_ip: 203.0.113.45
action: ssh_login_failed

Traceable
AUTH-L342 means line 342 of the login log

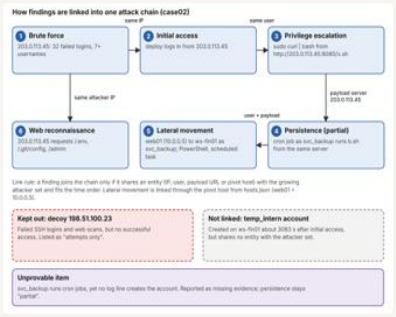
Nothing dropped
A line we cannot parse is kept and counted

Clock-aware
Cross-machine links allow a 5-minute gap

STEP 3 · LINK THE CHAIN

Linking findings into one story

1. Start with the address that guessed passwords and got in
2. Add findings that share a clue: the same user, link or second machine
3. Keep the decoy and unrelated account out
4. Report missing evidence instead of guessing



STEP 2 · FIND ATTACKS

Six simple rules, no AI

Rule	It fires when	Default threshold
Brute force	Many failed logins from one address	8 failures in 10 minutes
Initial access	A login succeeds soon after the guessing	Within 30 minutes
Privilege escalation	Admin command that downloads and runs code	Risky command patterns
Persistence	New account or scheduled job after access	Risky command patterns
Lateral movement	Hacked machine logs into a second one	Within 15 minutes
Web reconnaissance	Many page requests, mostly "not found"	15+ requests, 60% errors

No hard-coding: the attacker address, user and payload link are learned from the logs, never typed into the code.

WHAT THE SYSTEM FOUND

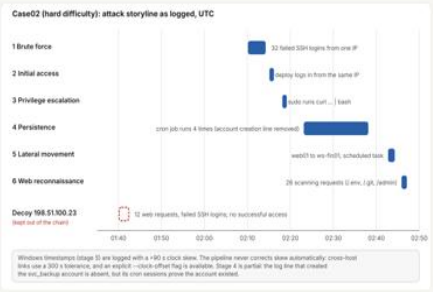
The attack on a timeline

Hard case, times in UTC

Six stages, about 37 minutes

The decoy has its own lane: it attacked but never got in

Windows events are logged 90 seconds late



STEPS 4 AND 5 · FACTS, THEN THE AI

The AI only rewrites proven facts

FACT FROM THE RULES

User deploy ran a sudo command on web01 that downloads and runs a script. **[AUTH-L330]**

SENTENCE FROM THE AI

The user deploy gained admin rights on web01 and ran a command that downloaded and executed a script. **[AUTH-L330]**

Small local model
qwen2.5:7b-instruct, free, runs on a laptop with Ollama

Same answer each time
Temperature 0 and a fixed seed

Strict rules
Add nothing new and cite the evidence IDs

STEP 6 · CHECK EVERY SENTENCE

Four layers of checking

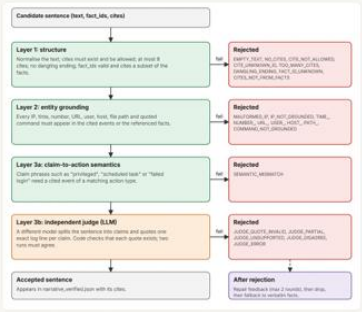
1 Structure: are the cited evidence IDs real and allowed?

2 Entities: does every IP, time and name appear in the cited lines?

3a Meaning: does the claim match the type of event cited?

3b Judge: a second AI quotes the exact log line for each claim

The final verdict is decided by code, not by the AI.



AGENT ORCHESTRATION

Eight agents, one controller

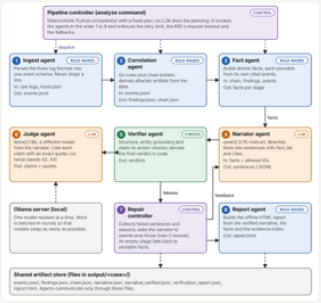
Controller: a fixed plan runs the agents in order and enforces the limits

Only two agents use an LLM: the Narrator and the Judge

Agents share files, not messages, so every hand-over can be inspected

Repair loop: failed sentences go back to the Narrator, at most two rounds

On purpose: a fixed plan, not a model choosing its own steps, so runs are repeatable



STEP 6 · REPAIR LOOP

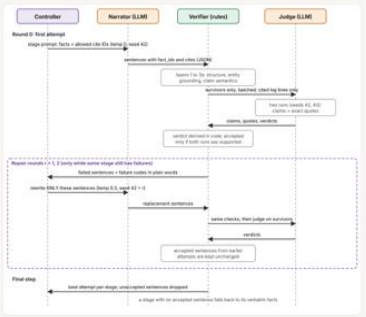
Fix what failed, keep what passed

Round 0: the Narrator writes a first draft

Checks: the Verifier and the Judge test each sentence

Rounds 1 and 2: only failed sentences go back, with the reasons

Still failing? It is dropped, or replaced by the plain verified fact



THE RESULT

The final report

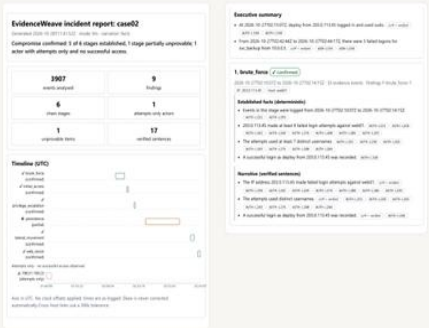
One offline file, easy to e-mail

Timeline with a separate lane for the decoy

Facts and verified sentences side by side

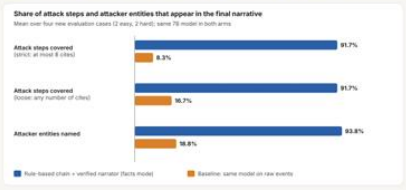
Click an evidence ID to see the raw log line

A section for what could **not** be proven



THE PROOF · WITH RULES VS WITHOUT

Same AI, very different results



Blue: rules + AI + checker

Orange: the same AI on raw logs only

The raw-log version also labelled routine jobs as threats, and was about 4× slower with 10× more tokens before any checking.

THE PROOF · RESULTS

Three numbers to remember

329/329
Attack lines found
On four new test cases, with no false additions

91.7%
Attack steps in the story
Same AI model on raw logs without rules: only 8.3%

119/120
Fake sentences rejected
Deliberately corrupted sentences the checker had never seen

Test cases come from the same generator as the rules, so this shows the rules work across its variations, not on real logs yet.

HONEST LIMITS

What the results do not show

Synthetic data
No real-world logs tested yet

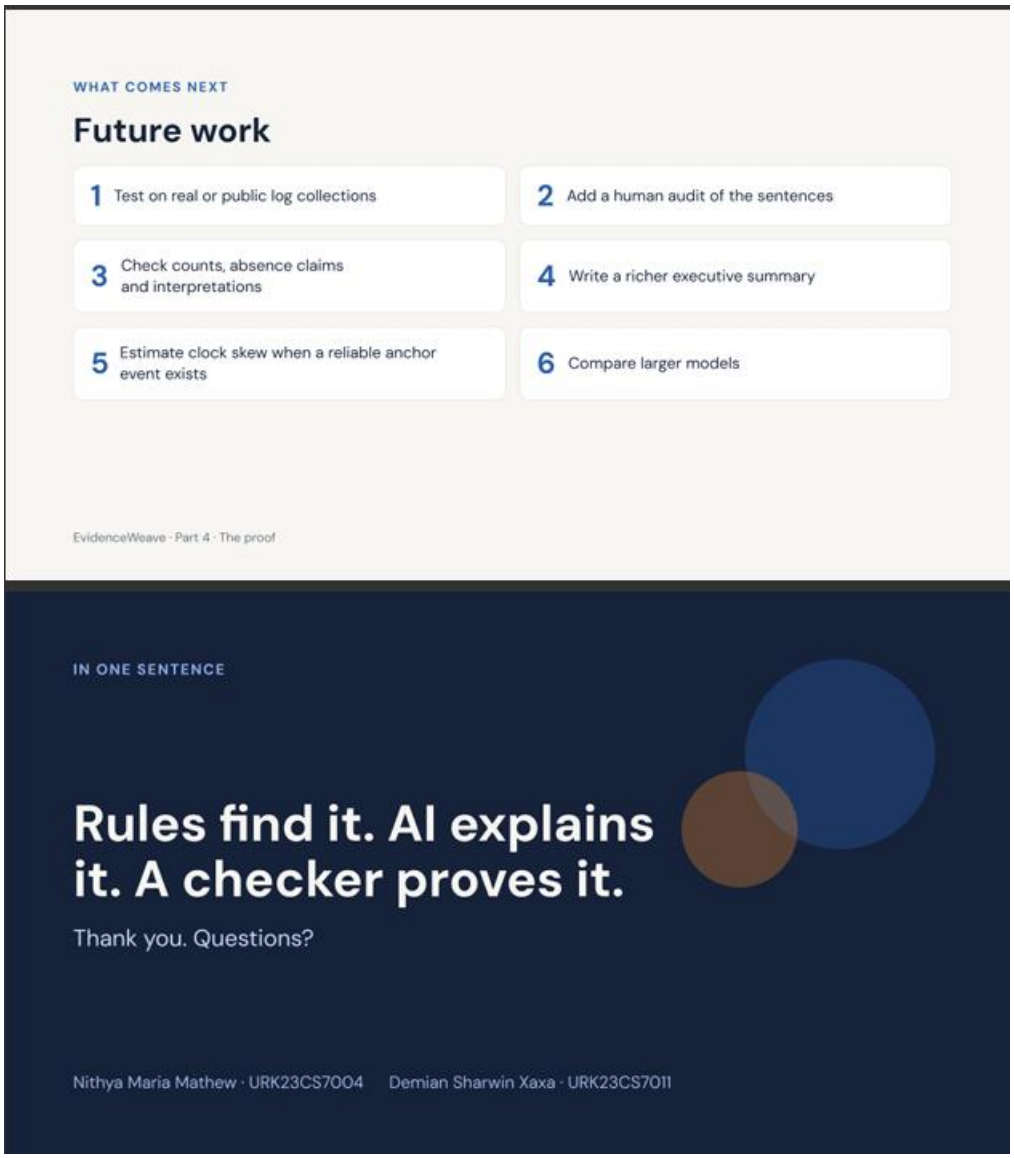
AI-labelled audit
The 32-sentence audit was labelled by an LLM, so it is not independent

Small samples
Four cases, one run each

One miss
1 of 120 corrupted sentences slipped through: a claim with no checkable name

False rejections
2 of 28 good sentences were wrongly rejected

Slow checking
A full run with the judge takes about 17 minutes


WHAT COMES NEXT
Future work
1 Test on real or public log collections
2 Add a human audit of the sentences
3 Check counts, absence claims and interpretations
4 Write a richer executive summary
5 Estimate clock skew when a reliable anchor event exists
6 Compare larger models
EvidenceWeave · Part 4 · The proof
IN ONE SENTENCE
Rules find it. AI explains it. A checker proves it.
Thank you. Questions?
Nithya Maria Mathew · URK23CS7004
Demian Sharwin Xaxa · URK23CS7011

6. CERTIFICATION

